

Name	
Pass	
Login	

Home

[Features](#)
[Screenshots](#)
[Clips](#)
[Requirements](#)
[Download](#)
[FAQ](#)
[Next version](#)
[Feedback](#)

Documentation

[Getting started](#)
[GUI Introduction](#)
[Basic editing](#)
[Advanced editing](#)
[Shortcut keys](#)
[Scripting API](#)
[Scripting API \(1.7\)](#)
[Scripting examples](#)

Technical

[GUI templates](#)
[Color bin files](#)
[Part bin files](#)
[Templates and donors](#)
[LDCad metas](#)
[Shadow library](#)
[Config files](#)

LDCad metas

On this page you'll find information and the full syntax of all LDCad specific LDraw meta line types. Some generic rules about LDCad metas are:

- They all start with **0 !LDCAD**
- They all have at least one or more subtype (keywords) directly following !LDCAD
- The remaining text are (optional) parameters in the **[option=value]** format separated by a space.

The value part of meta parameters comes in a variety of types, namely:

Type name	Default	Description
float	0.0	non whole number always uses '.' as decimal character no matter the os country setting.
integer	0	Whole number.
boolean	false	must be true or false
string		Freely given text. Allows for any non line breaking character except '[' and ']'. Must be one of a given collection of short strings depending on the meta using it.
enum		
ldrawRef		A reference to another LDraw file using the normal LDraw search method / locations etc. LDCad will automatically convert '\' and or '/' characters to the platform specific ones upon using the file name, but will always write the value using '\' ones.
ldrawColNr	16	32 bit unsigned integer indicating a LDraw color number.
relFileName		A file name relative to the containing LDraw file's location. LDCad will automatically convert '\' and or '/' characters to the platform specific ones upon using the file name. A specific variant might be forced while saving though (e.g. for LDraw files).
unit	0	float with optional unit indication. If no unit is specified LDraw coordinates will be used unless otherwise stated. Supported units are: mm : millimeter %: a percentage of another value. Can only be used if the property value allows for it. If so it will also describe what value the percentage is used on.
typeName array	n type defaults	N number of values separated by the space ' ' character. Arrays are often used to load a fixed number of options in one go. The order of

typeName list	n type defaults	values can therefore be very specific. N number of values separated by a comma ',' character. No real difference with array values except each value in the sequence is considered a standalone 'record' so removing any value will not change the meaning of the combined value.
LDraw matrix	0 0 0 1 0 0 0 1 0 0 0 1	LDraw style position vector and 3x3 matrix notation to be used to construct a 4x4 matrix.
3x3 matrix	1 0 0 0 1 0 0 0 1	Orientation matrix in the same order as the last nine numbers of a LDraw matrix.
vector	0 0 0	X, Y and Z float values to be used as a vector.

The default is only used / applied if the parameter using it hasn't got one of it's own.

Tool metas

The marker meta

Markers can be used to help align weirdly shaped things. You can also use them to help rotate selections.

Example

```
0 !LDCAD MARKER [posOri=-50 -24 0 1 0 0 0 1 0 0 0
1] [lengths=30 30 30 30 30 30]
```

Properties

Property	Type	Default	Description
posOri	LDraw matrix		Placement matrix
lengths	integer array	30 30 30 30 30 30	neg x, pos x, neg y, pos y, neg z an pos z axis lengths.
isTemp	bool	false	if true the marker will be automatically removed upon next save.
name	string		The optional name for this marker.

Grouping metas

LDCad supports nested recursive grouping of LDraw content. This means you can create groups of parts and or other groups even if they are spread among multiple (recursively used) LDraw files. To make this possible LDCad maintains several grouping related meta lines in models using groups.

The group definition meta

This meta will define the presence of a group inside a LDraw file. Each group must have a group definition meta in every LDraw file containing group member items. This meta should only be used in the header of LDraw files.

Example

```
0 !LDCAD GROUP_DEF [topLevel=true] [LID=0]
```

[GID=EjSe9B4luQa] [name=Group 1] [center=0 12 0]

Properties

Property	Type	Default	Description
topLevel	boolean	<i>true</i>	Indicates if this is the group's top level LDraw file or not.
LID	integer	<i>0</i>	Integer number to uniquely identify this group definition meta within this LDraw file. This is used to keep the item references (e.g. in GROUP_NXT lines) short by not using the full GID multiple times.
GID	string		Pseudo globally unique identifier of the LDCad group this meta references to. This id needs to be globally unique as groups can be spread amongst multiple files.
name	string	<i>Group n</i>	User supplied group name, must be unique among all top level groups in this LDraw file. Default will be generated like 'Group 1', 'Group 2' etc. Only used if toplevel is set to true.
center	vector		Relative center to use for this group. Only used if toplevel is set to true.

The group next meta

The group next meta is used to indicate the LDraw line following this meta line should be added to one or more LDCad groups. If the next line is not supported in LDCad grouping the meta it self will be ignored.

Example

0 !LDCAD GROUP_NXT [ids=0] [nrs=2]

Properties

Property	Type	Default	Description
ids	integer list		A LID reference to a local group's definition meta for each group this item is a member of.
nrs	integer list		The index of this item inside the referenced group's member list for each value of the ids list. These indexes are only used for sorting during loading so the used numbers might not be the same after the next save. Should have the same amount of elements as the id property.

The group object meta

The group object meta is used to group non saved LDCad specific content (e.g. auto groups like the path start and end points). It is almost identical to the GROUP_NXT meta but should only be used in the header of LDraw files.

Example

0 !LDCAD GROUP_OBJ [ids=0] [nrs=2] [refName=Path start]

Properties

Property	Type	Default	Description
ids	integer list		See GROUP_NXT

nrs	integer list	See GROUP_NXT
refName	enum	Name of the referenced object.

Content (management) metas

LDCad maintains a number of metas to keep track of extra LDCad specific information inside a LDraw (sub) file.

The script meta

The script meta is used to link a lua script file with a LDraw (sub)file. This meta must only be used in the header and currently only a single instance is allowed per subfile.

Example

```
0 !LDCAD SCRIPT [source=5580.lua]
```

Properties

Property	Type	Default	Description
source	relFileName		Relative file name of the lua script file you want to use with this model.

The content meta

The content meta is used to inform LDCad it should treat the LDraw content of this subfile in a special (non model) way. Its list of properties depend on the kind of content (indicated by the type property). This meta must be used in the header.

Example

```
0 !LDCAD CONTENT [type=path] [addFallback=default]
[looped=false] [displayKind=mm] [curveStep=0]
[displayLenCor=0]
```

Properties

Property	Type	Default	Description
type	enum		The content type must be either path or spring .
addFallback	enum		Must be one of the following: default : use the content/file location depended default. never : don't add or remove fallback content to / form the file. always : update / add fallback code whenever the file is saved. This is the default for normal locations. remove : don't add fallback content and remove any existing code when saving. This is the default for template locations.
Path specific properties			
looped	boolean	<i>false</i>	Indicates if the path should loop upon it self or maintain a start and end point.
displayKind	enum	<i>mm</i>	Used to set the display unit, must be one of the following: mm : millimeters.

			segs: segments. links: links (uses same value as segments).
curveStep	float	<i>0.0</i>	Used to set the internal curve resolution. Changing this value is only useful when constructing things made out of static non bendable segments, e.g. a technic chain. In such a case setting this value to the length of the segment part will result in perfect placement of e.g. the chain links. Otherwise you might see placement problems as the internal generator rounds everything to n times the resolution length which might slight differ from the segment curve length. 0.0 indicates the default should be used.
displayLenCor	integer	<i>0</i>	Correction to be applied when calculating the total path length. Must be given in the displayKind unit. Could be used to e.g. add the length of the cap parts to the generated path length.
Spring specific properties			
cRes	integer	<i>16</i>	Cylindrical resolution of the coil to use for (OpenGL) rendering.
cResLD	integer	<i>16</i>	Cylindrical resolution of the coil to use in the generated LDraw mesh.
cDia	unit	<i>8.0</i>	Inner diameter of the coil.
cRoll	float	<i>0.0</i>	Roll around the Y-axis to be applied to the coil in degrees.
wRes	integer	<i>8</i>	Cylindrical resolution of the wire to use for (OpenGL) rendering.
wResLD	integer	<i>8</i>	Cylindrical resolution of the wire to use in the generated LDraw mesh.
wDia	unit	<i>2.0</i>	Diameter of the wire.
wCol	ldrawColNr	<i>16</i>	Wire color
sepMesh	boolean	<i>false</i>	Indicates if the wire should be separated from the main spring content even if it's using color 16. If false it will only be separated if a non color 16 is used.

The generated meta

The generated meta is used to indicate all content following is maintained by one of LDCad fallback generators. This means that whole section of LDraw code might be replaced / removed when saving the file inside LDCad. It is therefor not recommended to manual edit the content in question.

Example

```
Ø !LDCAD GENERATED [generator=LDCad 1.4b]
```

or the slave variant:

```
Ø !LDCAD GENERATED [generator=LDCad 1.4b]
[master=pump.ldr]
```

Properties

Property	Type	Default	Description
----------	------	---------	-------------

generator	string		Version string of the program which created the following content last. The string is basically just FYI as it currently not used during loading at all.
master	ldrawRef		If this generated block is needed by another generated block (e.g. the spring mesh part of shock absorber spring content.) a reference to the master content must be given in this parameter. If a master is set this whole subfile will be managed by the content generator of the manager meaning it might be completely removed including any lines (up to and including 0 FILE) above the generator meta it self. The master content must reside in the same MPD.

Path content metas

Path content metas are used to define bendable parts inside a dedicated subfile. Based upon a collection of **0 !LDCAD PATH_** lines LDCad will be able to generate many kinds of flexible parts for use in your models. In order to also view those models correctly in other LDraw viewer/editor software a generated meta can be used to add plain LDraw fallback code to the subfile.

The path point meta

The path point meta is used to define a point the generated path must pass through in a certain way. If multiple point metas are present the path will follow the order of line placement.

Point metas must only be used in the header and optionally kept grouped together.

Example

```
0 !LDCAD PATH_POINT [type=bezier] [posOri=-110 0 0
1 0 0 0 1 0 0 0 1] [prevCPDist=25] [nextCPDist=25]
[cirR=25] [cirDir=xyCW] [prevYRoll=0]
[nextYRoll=0]
```

Properties

Property	Type	Default	Description
type	enum	<i>bezier</i>	Sets the point type, must be either bezier or circle . Bezier points are used to guide the path through the center of this point using two control points, one in front and one behind it using the y axis of the points coordinate system (given by posOri). Circle points are used to guide the path around the center of this point on a 2D plane using a custom radius.
posOri	LDraw matrix		Position and orientation of this point. The orientation part is very important as it guides the path's direction using its local Y-axis.
prevCPDist	float	25.0	Defines the control point 'in front of' this point by setting the distance (over the local Y-axis) to it in relation to the point's center. This point's previous control point will be used by the previous path point next control point in order to do the bezier calculations for the line segment between them.
nextCPDist	float	25.0	Defines the control point 'next to' this point by setting the distance (over the local Y-axis) to it in relation to the point's center. This point's next control point will be used together with the previous control point of the next path point to do the bezier calculations for the line segment between them.

cirR	float	25.0	Sets the radius of the path guidance circle. Only used if the point type is circle .
cirDir	enum	xyCW	Used to select the 2D plane and winding direction this circle path point must use to guide the path around its center. Must be one of the following: xyCW : Clockwise around the local xy plane. xyCCW : Counter clockwise around the local xy plane. zyCW : Clockwise around the local zy plane. zyCCW : Counter clockwise around the local zy plane. Only used if the point type is circle .
prevYRoll	float	0.0	Sets the roll (in degrees) around the paths Y-axis the path donor is required to have before following the local circle arc. Needed when e.g. working with a square shaped rubber band to guide it from e.g. a bush onto a beveled wheel. Only used if the point type is circle .
nextYRoll	float	0.0	Sets the roll (in degrees) around the paths Y-axis the path donor is required to have before leaving the local circle arc. Needed when e.g. working with a square shaped rubber band to guide it from e.g. a beveled wheel onto a bush. Only used if the point type is circle .

The path cap meta

The path cap meta is used to add static (sub)parts to a flexible part. This is usually reserved for the outer endings of the generate part e.g. the studs at both endings of a wire.

Cap metas must only be used in the header and optionally kept grouped together.

Example

```
0 !LDCAD PATH_CAP [group=start] [color=16]
[posOri=-170 8 0 1 0 0 0 1 0 0 0 1] [part=165.dat]
[extraLen=0mm]
```

Properties

Property	Type	Default	Description
group	enum		Group is used to assign this cap to one of the automatic LDCad groups available for this flexible part. It must be set to one of the following: start : All things that make up the starting point of a flexible part. This is usually a cap and the first path point. end : All things that make up the end point of a flexible part. This is usually a cap and the last path point.
color	ldrawColNr	16	Color to use given LDraw part with.
posOri	ldraw matrix		Position and orientation of the cap's part.
part	ldrawRef		LDraw part to use as a cap.
extraLen	unit	0mm	Can be used to state the captions length, can be used to corrected the total path length when the caption parts include e.g. part of the flexible donor structure themselves.

The path anchor meta

The anchor meta is (optionally) used to force a custom center using the automatic path content groups. This might be needed for **start** and or **end** groups if the containing cap or path point isn't suitable or logically. There can only be one anchor meta per group type inside any path content subfile.

Anchor metas must only be used in the header and optionally kept grouped together.

Example

```
0 !LDCAD PATH_ANCHOR [group=start] [posOri=-110 24
0 1 0 0 0 1 0 0 0 1]
```

Properties

Property	Type	Default	Description
group	enum		Group is used to assign this anchor to one of the automatic LDCad groups available for this flexible part. It must be set to one of the following: start : All things that make up the starting point of a flexible part. This is usually a cap and the first path point. end : All things that make up the end point of a flexible part. This is usually a cap and the last path point. It is not allowed to set group to something another anchor is already set to.
posOri	ldraw matrix		Position and orientation of the anchor.

The path skin meta

The path skin meta is used to define how to fill/flesh out (part of) the path taken through the given point(s). There should be at least one skin meta in every path content subfile. It is allowed to use multiple skin metas. The total available path length will be divided amount multiple skin metas depending on their **segments (seg*)** related properties.

Skin metas must only be used in the header and optionally kept grouped together.

Example

```
0 !LDCAD PATH_SKIN [donCol=16] [donOri=1 0 0 0 1 0
0 0 1] [donPart=3001.dat] [donYSize=100%]
[donCen=absCen] [donCenYOfs=0]
[donFinScale=fitDon2Seg] [donPlace=refsDyn]
[donYAlign=0] [donInline=false] [segSize=100%]
[segSizeTol=5%] [segsCnt=0] [segsGrp=0]
[segsMaxMerge=1] [segsMrgAng=0]
[segsEdgeDelKind=keepFirstLeft]
```

Properties

Property	Type	Default	Description
donCol	ldrawColNr	16	Color to use with the given donor part.

	donOri	3x3 matrix		Rotation of the donor part. donOri act as a correction of the donor's own orientation in order to make the part Y-axis oriented. This because the donor part will be 'smeared' along the path in its local (after this correction) Y direction.
	donPart	ldrawRef		Set the LDraw file to use for filling out the path. Any LDraw file is allowed here but in practice you want to limit usage to simple parts.
	donYSize	unit	<i>100%</i>	This is used to set the base donor size in its Y direction. If for example the donor part is a tube section with a height of 2ldu could stretch (y-axis only) that using this option to e.g. 10ldu en large it times e..g 4 by using 400%.
	donCen	enum	<i>absCen</i>	Used to choose the working center of the donor part. This option uses the donor as it is after the orientation corrections. It must be one of the following: partCen : Use the part origin. absCen : Use the absolute center of the part. absYCen : Use the part origin for the X and Z axis, but use the absolute center of its Y-axis. absXZCen : Use the absolute center of the X and Z axis, but use the part origin for the Y-axis.
	donCenYOfs	float	<i>0.0</i>	Define an additional offset to the Y part of the donor (after donOri, donYSize and donCen corrections) to be used.
	donFinScale	enum	<i>fitDon2Seg</i>	Controls how the donor is scaled before placement in a segment. It must be set to one of the followeling: none : No additional scaling. fitDon2Seg : Stretch/shrink the donor result to exactly fit the destination segment. Do note this might (partly) override some of the other don* scaling related parameters which could result in an unexpected end result.
	donPlace	enum	<i>refsDyn</i>	Selects the placement / generation method. Must be one of the following: refsStat : Donors are placed using the calculated center (after donYAling correction) at the starting point of segments using that locations local path orientation. refsDyn : Donors are placed using the calculated center (after donYAling correction) at the center segements using that locations (interpolated) local path orientation. deform : The donor mesh is deformed in such a way it follows the path's curve as a result any fallback code will have the full recursive (transformed) source of the donor part for each segment it is used on. The deform

				method results in extremely smooth flexible parts but it might also result in very large LDraw files.
	donYAlign	integer	<i>0</i>	Sets an alignment correction to use on the calculated donor center before placing a donor at a path segment. A negative number will shift the donor to the left by multiples of its Y length. So -1 will move it a full length to the left. A positive number will do the same to the right. 0 means no change will be made.
	donInline	boolean	<i>false</i>	Determines if the generator will try to inline the donor file instead of referring to it directly while creating the segment fallback code. In-lining the donor file removes dependency on it and might increase portability of the fallback code. The donor will only be in-lined if it is entirely made out of type 1 lines.
	segSize	unit	<i>100%</i>	Controls the length of individual segments along the resulting path. Each segment on the path will be filled out with a single donor part instance. Setting the segment size therefore controls how many times the donor might be used to fill out the path. Be care full when setting the segment size while using the deform method as it will greatly influence the resulting fallback file size. You can supply the segment size in ldu, mm or as a percentage of donYSize.
	segSizeTol	unit	<i>5%</i>	Controls the amount of scaling allowed upon segments in order to fill rest space. If for example the segments size is 10 and the whole path has a length of 111, resulting in 11 segments, the remaining length of 1 will be spread over the 11 segments by scaling them slightly. Use this option to limit the amount of scaling to prevent e.g. noticeable distortions in the donor part. tolerance can be given in a static ldu, mm or a percentage of the calculated segment size it self.
	segsCnt	integer	<i>0</i>	Used to set optionally set a fixed number of segments. 0 means unlimited segments can be used in order to construct the path. If a non zero value is given only a portion of the path will be filled out using this skin meta's options. This option can be used to limit the visible length of a flexible part and make room for other skin metas on the path.
	segsGrp	integer	<i>0</i>	Used to group segments resulting from different skin metas. If set to 0 no grouping will be done and each skin meta will fill out its assigned space in order of the skin meta lines. If set to non zero skin metas using the same group number will alternate (zip together) their segments making possible e.g. color patterns.
	segsMaxMerge	integer	<i>1</i>	In order to try and reduce the fallback code segments which are in a straight line of each other could optionally be

segsMrgAng	float	<i>0.0</i>	joined into a single segment resulting in less donor placements. this option is only usefull when the donor uses final scaling other wise you will end up with (large) gaps in your path. optionally lets you define the maximum angle (in degrees) between two segments in order to deem them to be inline of each other. 0.0 indicates the internal (version depended) hard coded default should be used. Use this option with case as a large angle might cause alignment problems.
segsEdgeDelKind	enum	<i>keepFirstLeft</i>	Controls what to do with overlapping conditional lines while using the deform placement method. Must be one of the following: keepFirstLeft : Remove the left ones except for the first segment. keepLastRight : Remove the right ones except for the last segment. keepAll : Remove nothing, keep them all.

The path length meta

The path length meta is used to constrain the length of the path and or to define a real life (official library) counter part for the maximum length. When multiple length metas are present the constraint closest to the true length of the path will be applied in order to e.g. map path content to static LDraw files.

Length metas must only be used in the header and optionally kept grouped together.

Example

```
0 !LDCAD PATH_LENGTH [lenTarget=50mm]
[LenMargin=2%] [partName=some.dat] [partDescr=50
mm pneuhose]
```

Properties

Property	Type	Default	Description
lenTarget	unit	<i>0</i>	The maximum allowed path length this meta can be used for.
lenMargin	unit	<i>0</i>	The amount a path's length might differ from the true constrains in order to still qualify. This helps prevent unwanted exclusions and compensates for stretchable parts (e.g. rubber bands). The margin can be given in ldu, mm or a percentage upon the lenTarget value.
partName	ldrawRef		LDraw part which represents the static version of a path which matches this metas constrains. Currently this reference is never loaded so it does not have to exists. future versions of LDCad might use it in e.g. part usage overviews etc.
partDescr	string		Description to use for the given partName. Currently nothing is done with this value but future versions of LDCad might use it when e.g. the partName LDraw file can not be found (is virtual)

Spring content metas

Spring content metas are used to define parts which include a movable spring inside a dedicated subfile. Based upon a collection of **0 !LDCAD SPRING_** lines LDCad will be able to generate many kinds of spring oriented parts for use in your models. In order to also view those models correctly in other LDraw viewer/editor software a generated meta can be used to add plain LDraw fallback code to the subfile.

The spring point meta

The spring point meta is used to define at most two points using different group settings. These are the points the generated spring will connect using one or more section metas.

Point metas must only be used in the header and optionally kept grouped together.

Example

```
0 !LDCAD SPRING_POINT [group=start] [posOri=0 24 0
1 0 0 0 1 0 0 0 1]
```

Properties

Property	Type	Default	Description
group	enum		Group is used to assign this point to one of the automatic LDCad groups available for spring parts. It must be set to one of the following: start : This point represents the start or bottom of the spring. end : This point represents the end or top of the spring. It is not allowed to set group to something another point is already set to.
posOri	LDraw matrix		Position and location of this spring point.

The spring cap meta

The cap meta is used to add static part(s) to either the start or end of the spring oriented part.

Example

```
0 !LDCAD SPRING_CAP [group=start] [color=16]
[posOri=0 24 0 1 0 0 0 1 0 0 0 1] [part=3005.dat]
```

Properties

Property	Type	Default	Description
group	enum		Group is used to assign this cap to one of the automatic LDCad groups available for spring parts. It must be set to one of the following: start : This cap connects to the start or bottom of the spring. end : This cap connects to the end or top of the spring.
color	ldrawColNr	16	Color to use for this cap part.
posOri	LDraw matrix		Position and location of this cap part.
part	ldrawRef		LDraw part to use for this cap. Can be anything but in practice it should be a single LDraw part as it might hurt render performance otherwise.

The spring anchor meta

The anchor meta is used to force a custom center (and orientation) for one of the spring related groups. There can only be one anchor meta per group type in any spring content subfile.

Anchor metas must only be used in the header and optionally kept grouped together.

Example

```
0 !LDCAD SPRING_ANCHOR [group=start] [posOri=0 48  
0 1 0 0 0 1 0 0 0 1]
```

Properties

Property	Type	Default	Description
group	enum		Group is used to assign this anchor to one of the automatic LDCad groups available for spring parts. It must be set to one of the following: start : This cap connects to the start or bottom of the spring. end : This cap connects to the end or top of the spring. It is not allowed to set group to something another anchor is already set to.
posOri	LDraw matrix		Position and location of this anchor.

The spring section meta

The section meta is used to define the appearance of (part of) the spring wire connecting the two spring points. At least one section meta should be present. When multiple section metas are given the available spring length will be divided among them based on the **proportion** property.

Section metas must only be used in the header and optionally kept grouped together.

Example

```
0 !LDCAD SPRING_SECTION [windingCnt=5]  
[proportion=1]
```

Properties

Property	Type	Default	Description
windingCnt	float	5.0	Number of windings around the springs' Y-direction this meta should make within the available length.
proportion	integer	1	Defines the portion of the total spring length this section controls. If set to 0 the wire will be wound tightly and this section's length depends on the number of windings and wire thickness. If set to non zero the available length will be calculated by taking the leftover length (spring length minus the length used by zero proportion sections) divided by the sum of all proportion values multiplied by this meta's proportional value. For example imagine having a spring of length 100 using 4 section metas. If one of those sections uses a proportion of 2 and the other three use 1 the resulting section lengths will be: 40 (2 5ths), 20 (1 5th), 20 and 20.

Part snapping metas

Part snapping metas are currently exclusively used combined with shadow library files in order to add extra information to LDraw part files. This information is needed by LDCad to calculate possible 'LEGO like' part placement positions.

These metas basically describe hotspots and their shapes which then will be used to test against each other while adding a new part to the current model. This will be done for all snap info on both the source and destination brick(s) in order to find the best / closest match (e.g. an axle pin into a beam hole).

The snap clear meta

The clear meta is used to flush all or part of the inherited snap information gathered for the current part file so far. This meta might be needed to replace inherited information in order to extend it or increase its detail. Like e.g. information obtained from an axle primitive because that primitive is used as part of a bigger shape inside this file.

Example

```
0 !LDCAD SNAP_CLEAR [id=axleHole]
```

Properties

Property	Type	Default	Description
id	string		Used to only clear information obtained from snap info using the given freely named ID. Leave it empty to clear all information for the current part.

The snap include meta

The include meta is used to add the information from another shadow library file to this part too. This is done non recursively.

Example

```
0 !LDCAD SNAP_INCL [ref=connhole.dat] [pos=-50 10  
0] [ori=0 -1 0 0 0 -1 1 0 0] [grid=C 1 C 3 20 20]
```

Properties

Property	Type	Default	Description
ID	string		Optional identifier which can be used in clear metas to optionally drop this meta's information in higher level parts using it.
pos	vector		Position of the included information / object.
ori	3x3 matrix		Orientation transformation to apply to the included information / object.
scale	vector		Optional scaling vector to apply to the included information / object
ref	ldrawRef		Reference to the source shadow library file. Must only use local part references.
grid	mixed array		Defines a grid pattern to use for multiple placement / inclusion of the referenced file. The grid uses the orientation stated in the ori parameter. As LDraw part data is Y-axis orientated only the X and Z grid stepping values need to be given like so: Xcnt Zcnt Xstep Zstep for example: 4 8 20 20 which could be used to make a 4x8 grid of e.g. studs. Optionally each count value can be preceded by a C character indicating the grid should be centered on

that axis. If no C is given the axis will add to the pos parameter. For example to center the 4x8 grid around its pos parameter use: **C 4 C 8 20 20**

The snap cylinder meta

The cylinder meta is the main workhorse among the snap info metas. This because it describes holes and pens which is what LEGO is all about. If you take a good look at some LEGO bricks you'll notice all of them have similar interlocking holes and pens. The only real difference between them are the diameters and pattern of the hole/pens shapes (like the side of a key). It is this information that the cylinder meta tries to capture in order to calculate matching pairs.

Example

```
0 !LDCAD SNAP_CYL [id=connhole] [gender=F]
[caps=none] [secs=R 8 2 R 6 16 R 8 2]
[center=true] [slide=true] [pos=0 0 0] [ori=1 0 0
0 1 0 0 0 1]
```

Properties

Property	Type	Default	Description
ID	string		Optional identifier which can be used in clear metas to optionally drop this meta's information in higher level parts using it.
group	string		optional group identifier. Can be used to limit potential matches to only snap info having the same group string. Can be used to prevent unwanted matches when very complicated shapes are involved e.g. click rotation holes etc.
pos	vector		Position of this shape.
ori	3x3 matrix		Orientation of this shape.
scale	enum	none	Defines how scaled references to the master (official) part should be handled information inheritance wise. Must be one of the following: none : If scaling is detected this information will not be inherited by the higher level part. YOnly : The information will only be inherited if scaling is limited to the Y-axis, if X and or Z is scaled the info will not be inherited. ROnly : The information will only be inherited if scaling is limited to the cylinder's radius (usually x and z) given its done symmetrical. If the info is scaled in any other way it will not be inherited. YandR : The information will only be inherited if YOnly or ROnly rules apply.
mirror	enum	cor	Defines how mirrored references to the master (official) part should be handled information inheritance wise. Must be one of the following: none : If mirroring is detected this information will not be inherited by the higher level part. cor : If mirroring is detected the snap information will be corrected by flipping one of the radius axis'.
gender	enum	male	Sets the gender of the cylinder shape M for male (pen) and F for female (hole).
secs	mixed array		Describes the shape of the hole (along the neg Y-axis) or pen by a sequence of shape variants, radius's and lengths. The info must be given in blocks of:

			shapeVariant radius length where shapeVariant must be one of the following: R: Round. A: Axle. S: Square. _L: Flexible radius wise extension to the previous block's specs. This will be needed for e.g. the tip of an technic connector pin. Although it is slightly larger it allows for (temporary) compression while sliding the pin inside e.g. a beam hole. L_: Same as _L but as an extension to the next section instead of the previous one. For example a plain stud can be described using a single block: R 8 4 while a technic beam hole needs three: R 8 2 R 6 16 R 8 2.
caps	enum	one	Defines the ends of the shape, must be one of the following: none: The shape is open ended. e.g. a male axle or female beam hole. one: The shape has one closed ending, which one depends on the gender. For male shapes it will be A (bottom) and for female shapes it will be B (top). two: The shape is closed (blocked) at both sides. e.g. the male bar of a minifig suitcase handle. A: The bottom is closed / blocked. e.g. a stud. B: The top is closed / blocked. e.g. an anti stud.
grid	mixed array		Defines a grid pattern to use for multiple placement of this cylindrical shape. The grid uses the orientation stated in the ori parameter. As all snap info is Y-axis orientated only the X and Z grid stepping values need to be given like so: Xcnt Zcnt Xstep Zstep for example: 4 8 20 20 which could be used to make a 4x8 grid of e.g. studs. Optionally each count value can be preceded by a C character indicating the grid should be centered on that axis. If no C is given the axis will add to the pos parameter. For example to center the 4x8 grid around it's pos parameter use: C 4 C 8 20 20
center	boolean	false	Indicates if this cylinder shape should be centered at its position or not.
slide	boolean	false	Indicates if this cylinder shape should be considered 'smooth' enough to make sliding of matching parts possible. If either part of a matched pair of snap info metas has the slide option set to true the user will be able to slide them together. If not it will just 'snap'. Be careful while setting this option as it can cause unwanted sliding of e.g. a stud inside an anti stud. In practice it is best to limit the slide=true value to things you know will slide most of the time (e.g. clips, bush and gear parts etc).

The snap clip meta

The clip meta is used to describe all clip like shapes. Clips are always of the female gender and will be tested against male cylinder shapes.

Example

0 !LDCAD SNAP_CLP [radius=4] [length=8] [pos=0 0

0] [ori=1 0 0 0 1 0 0 0 1] [center=true]

Properties

Property	Type	Default	Description
ID	string		See cylinder meta.
pos	vector		See cylinder meta.
ori	3x3 matrix		See cylinder meta.
radius	float	4.0	Inner radius of the clip while in rest.
length	float	8.0	Length or width of the clip's (partial) hole.
center	boolean		See cylinder meta.
slide	boolean		See cylinder meta.
scale	enum	none	See cylinder meta.
mirror	enum	none	See cylinder meta.

The snap fingers meta

The fingers meta is used to describe all the interlocking finger like shapes. finger shapes will only be tested among them selves.

Example

0 !LDCAD SNAP_FGR [group=lckHng] [genderOfs=M]
[seq=4.5 8 4.5] [radius=6] [center=true] [pos=-30
10 0] [ori=1 0 0 0 0 1 0 -1 0]

Properties

Property	Type	Default	Description
ID	string		See cylinder meta.
group	string		See cylinder meta.
pos	vector		See cylinder meta.
ori	3x3 matrix		See cylinder meta.
genderOfs	enum	male	Indicates the gender of the first finger of the shape, must be ether M for male or F for female.
seq	float array		Defines the width of each finger by a sequence of floats.
radius	float		Outer radius of the finger (tip). these are usually beveled and thus the main snapping parameter.
center	boolean		See cylinder meta.
scale	enum	none	See cylinder meta.
mirror	enum	none	See cylinder meta.

The snap generic meta

The generic meta is used to describe very oddly shaped things which in practice only have a few counter parts. Things like electric plugs and window glass etc.

Matching is done only among generic metas themselves based on the group name alone. But the bounding information will influence the best candidate selection when multiple pairs are located closely together.

Example

0 !LDCAD SNAP_GEN [group=nxtc] [gender=M] [pos=0
-1.5 1.5] [ori=1 0 0 0 0 1 0 -1 0] [bounding=box
12.5 16.5 8]

Properties

Property	Type	Default	Description
ID	string		See cylinder meta.
group	string		See cylinder meta.
pos	vector		See cylinder meta.
ori	3x3 matrix		See cylinder meta.
gender	enum	male	See cylinder meta.
bounding	mixed array		Defines a bounding shape which will be used in the 'best pair candidate' calculations. Is needed because the positional alone will might cause unwanted matches otherwise. There are several types of bounding shapes, namely: pnt : Single point / no bounding box, no extra info needed. box : Box defined by its following x, y, and z radius', for example: box 10 8 10 cube : Cube defined by a following (half) size, for example: cube 8 cyl : Cylinder defined by a following radius and length, for example: cyl 8 20 sph : Sphere defined by a following radius, for example: sph 8
scale	enum	none	See cylinder meta.
mirror	enum	none	See cylinder meta.

The snap sphere meta

This meta will be replaced by the generic meta in LDCad 1.5 as that version introduces extra parameters for the generic meta making the sphere meta redundant.

The sphere meta is used to describe a spherical object like a ball joint. Although it has orientation matched pairs will be snapped together using their current orientation inside the editor as things like ball joints etc are moved freely. Future versions of this meta might optionally limit that freedom.

Example

0 !LDCAD SNAP_SPH [gender=M] [radius=4]

Properties

Property	Type	Default	Description
ID	string		See cylinder meta.
pos	vector		See cylinder meta.
ori	3x3 matrix		See cylinder meta.
gender	enum	male	See cylinder meta.
radius	float		Radius of the sphere.

scale	enum	<i>none</i>	See cylinder meta. But only none and ROnly are allowed.
mirror	enum	<i>none</i>	See cylinder meta.

Copyright Roland Melkert 2011–2017, all rights reserved. LEGO® is a registered trademark of the LEGO Group, which does not sponsor, endorse, or authorize this site.